

Department of Computer and Software Engineering**SE: Machine Learning****Course Instructor:****Date:****Day:****Semester:****Lab 11: Refactoring, MLflow and Deployment**

Lab Title	CLO	BT	PLO	Weightage
Refactoring & Deployment MLflow, APIs, Serialization				

Name	Roll Number	Report /10	Viva /5	Total /15
Muhammad Abdul Qadir	BSSE23105			

Checked on: _____

Signature: _____

Objectives

- Refactor unstructured ML code into modular functions
- Track experiments using MLflow
- Serialize trained models
- Build REST APIs using FastAPI

Problem Statement

In real-world machine learning systems, models are not built as isolated scripts but as part of structured, maintainable, and deployable pipelines.

You are given a poorly written (“spaghetti”) machine learning script that mixes data loading, preprocessing, model training, and evaluation in a single block of code. Such code is difficult to maintain, extend, debug, and deploy.

Your task is to transform this code into a clean and modular pipeline by separating concerns into well-defined functions.

After refactoring, you will extend the pipeline to include:

- **Experiment Tracking:** Use MLflow to log model parameters, performance metrics, and artifacts.
- **Model Serialization:** Save the trained model to disk and reload it for inference.
- **Model Deployment:** Build a REST API using FastAPI to serve predictions from the trained model.

You will use the dataset from the previous lab (Lab 10) to ensure continuity.

By the end of this lab, you will have a complete mini machine learning system that includes:

- Clean, modular code
- Experiment tracking
- Persistent models
- A working prediction API

Part A: Refactoring

Task 1: Identify Issues

Open `train_bad.py` and list at least four problems related to structure and maintainability (e.g. hard-coded paths, no functions, magic numbers, mixed concerns).

Issues identified in `train_bad.py`:

1. **No functions at all:** Everything runs as one big block of code from top to bottom. There are no function definitions, so we cannot test, reuse, or call any part of the code separately.
2. **Hard-coded file path:** The dataset name `"lab10_data.csv"` is written directly in the code instead of being passed as a parameter. If the file name changes, we have to go into the code and change it manually.
3. **Magic numbers:** The model is created with values like `max_depth=100` and `min_samples_split=2` without any explanation of why those numbers were chosen. This makes it hard for someone else to understand the reasoning.
4. **Mixed concerns:** Loading data, cleaning it, training the model, and printing results are all mixed together in one place. Ideally each step should be in its own function so the code is easier to follow and maintain.
5. **Dead and useless code:** There is an `unused_list`, an `unused_dict`, a `for` loop that does nothing, an `if True` block that always runs, and a line that multiplies every column by 1 which has no effect. All of this adds clutter without any benefit.

Task 2: Modularization

Refactor the script into the following functions inside `lab11_starter.py`:

- `load_data()` — read `lab10_data.csv` and return a `DataFrame`.
- `preprocess_data(df)` — drop the categorical `city` column, then split into feature matrix `X` (all columns except `target`) and label vector `y`.
- `train_model(X, y)` — fit and return a `DecisionTreeClassifier`.
- `evaluate_model(model, X, y)` — compute and return the accuracy score (expected ≥ 0.8).
- `run_pipeline()` — orchestrate the above steps and save the model to disk.

Refactored functions in lab11_starter.py:

```
def load_data():
    df= pd.read_csv("lab10_data.csv")
    return df

def preprocess_data(df):
    # drop city column since its categorical
    df = df.drop("city", axis=1)
    X= df.drop("target", axis =1)
    y = df["target"]
    return (X, y)

def train_model(X, y):
    clf= DecisionTreeClassifier(random_state=42)
    clf.fit(X, y)
    return clf

def evaluate_model(model, X, y):
    preds= model.predict(X)
    acc = accuracy_score(y, preds)
    return acc
```

```
(MLlabtestenv) PS D:\ML-Labs\BSSE23105_B_ML_LAB11> python lab11_solution.py
task 2 - modularization

loaded the dataset
rows: 1000, columns: 6
column names: ['age', 'income', 'city', 'hour', 'target', 'leak_feature']

preprocessed the data
X shape: (1000, 4)
y shape: (1000,)
features used: ['age', 'income', 'hour', 'leak_feature']

trained the decision tree
max_depth = None
criterion = gini

model accuracy: 1.0
this is above the threshold of 0.8, so we are good
```

Part B: MLflow

Task 3: Logging

Inside `run_pipeline()`, wrap the training and evaluation steps in an MLflow run and log the following:

- **Parameters:** model hyper-parameters (e.g. `max_depth`, `criterion`).
- **Metrics:** training accuracy (key: `accuracy`; must reach the expected accuracy threshold of 0.8).
- **Model:** the fitted `DecisionTreeClassifier` via `mlflow.sklearn.log_model`.

`run_pipeline()` with MLflow logging:

```
def run_pipeline():
    df = load_data()
    X, y = preprocess_data(df)

    mlflow.set_experiment("lab11_pipeline")
    with mlflow.start_run():
        mdl = train_model(X, y)
        acc = evaluate_model(mdl, X, y)
        # log params
        mlflow.log_param("max_depth", mdl.get_params()["max_depth"])
        mlflow.log_param("criterion", mdl.get_params()["criterion"])
        # log metric
        mlflow.log_metric("accuracy", acc)
        # log the model artifact
        mlflow.sklearn.log_model(mdl, "model")

    joblib.dump(mdl, "model.joblib")
    print(f"model saved. accuracy: {acc}")
    return acc
```

task 3 - mlflow logging

running the full pipeline now (with mlflow)...

model saved. accuracy: 1.0

pipeline done, final accuracy = 1.0

Part C: Serialization

Task 4: Save and Load Model

Use `joblib` to persist the trained model:

- In `run_pipeline()`: call `joblib.dump(model, "model.joblib")` after training.
- In the FastAPI section: call `joblib.load("model.joblib")` to restore the model before the app starts serving requests.

Saving the model in `run_pipeline()`:

```
# at the end of run_pipeline
joblib.dump mdl, "model.joblib")
```

Loading the model in the FastAPI section:

```
# load saved model if available
if os.path.exists("model.joblib"):
    model = joblib.load("model.joblib")
else:
    model = None
```

task 4 - serialization

testing if saved model works...

sample features: [56.0, 11406.0314, 3.0, 0.0047]

prediction: 0

actual label: 0

Part D: FastAPI

Task 5: API Endpoint

Complete the `/predict` POST endpoint in `lab11-starter.py`:

- **Request body:** `{"features": [age, income, hour, leak_feature]}`
- **Response:** `{"prediction": 0}` or `{"prediction": 1}`
- Raise `HTTPException(status_code=422)` if the `features` key is missing.
- Raise `HTTPException(status_code=503)` if the model has not been loaded.

Verify the API is reachable via the GET `/` endpoint which returns `{"message": "ML Model API is running"}`.

FastAPI endpoints:

```
app = FastAPI()

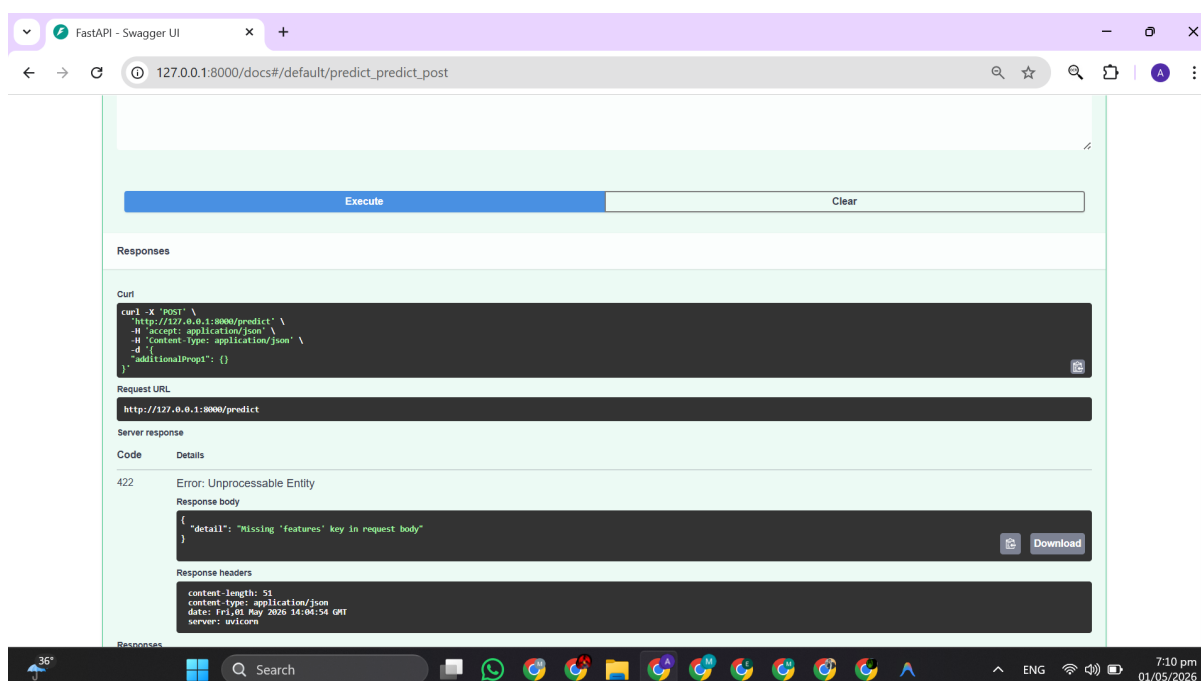
# load saved model if available
if os.path.exists("model.joblib"):
    model = joblib.load("model.joblib")
else:
    model = None

@app.get("/")
def home():
    return {"message": "ML Model API is running"}

@app.post("/predict")
def predict(data: dict):
    if model is None:
        raise HTTPException(status_code=503,
                            detail="Model not loaded")
    if "features" not in data:
        raise HTTPException(status_code=422,
                            detail="Missing 'features' key in request
                                    body")
    cols = ["age", "income", "hour", "leak_feature"]
    inp = pd.DataFrame([data["features"]], columns=cols)
    result = model.predict(inp)
    return {"prediction": int(result[0])}
```

The screenshot shows the Swagger UI for a FastAPI application. The browser address bar displays `127.0.0.1:8000/docs/#/default/home_get`. The interface is divided into sections: **Parameters** (No parameters), **Responses**, and **Server response**. The **Execute** button is highlighted in blue. Below the **Execute** button, the **Response body** is shown as a JSON object: `{ "message": "ML Model API is running" }`. The **Response headers** are also displayed: `content-length: 37`, `content-type: application/json`, `date: Fri, 01 May 2026 13:52:07 GMT`, and `server: uvicorn`. The Windows taskbar at the bottom shows the time as 6:56 pm on 01/05/2026.

The screenshot shows the Swagger UI for a FastAPI application. The browser address bar displays `127.0.0.1:8000/docs/#/default/predict_predict_post`. The interface is divided into sections: **Parameters** (No parameters), **Responses**, and **Server response**. The **Execute** button is highlighted in blue. Below the **Execute** button, the **Response body** is shown as a JSON object: `{ "prediction": 0 }`. The **Response headers** are also displayed: `content-length: 37`, `content-type: application/json`, `date: Fri, 01 May 2026 13:52:07 GMT`, and `server: uvicorn`. The Windows taskbar at the bottom shows the time as 7:11 pm on 01/05/2026.



Discussion Questions

1. Why is modular design important?

Modular design matters because it breaks the code into small, separate functions where each one does just one thing. This way, if something goes wrong, we can quickly find which function has the issue instead of reading through a huge script. It also makes testing easier since we can check each function on its own. On top of that, if we write a good `load_data()` function once, we can reuse it in other projects without writing it again. It also helps when working in a team because different people can work on different functions without getting in each other's way.

2. What are the benefits of MLflow?

MLflow helps us keep track of our machine learning experiments in an organized way. Every time we train a model, it saves the parameters we used, the accuracy we got, and even the model itself. This is really helpful because without it, we would lose track of which settings gave us the best results. MLflow also has a nice web dashboard where we can compare different runs side by side. Another benefit is that it works well with libraries like scikit-learn, so we only need to add a few lines of code to start using it.

3. Why is model serialization required?

Model serialization means saving a trained model to a file so we can load it later without training it again. This is important because training can take a lot of time, especially with big datasets. Once we save the model using something like `joblib`, we can just load it whenever we need to make predictions, for example in an API. It also lets us share the model with others or move it to a different machine. Without serialization, we would have to retrain the model every time we restart the program, which is not practical at all.

Test Results

```
(MLlabtestenv) PS D:\ML-Labs\BSSE23105_B_ML_LAB11> pytest -v lab11_tests.py
===== test session starts =====
platform win32 -- Python 3.10.19, pytest-9.0.2, pluggy-1.5.0 -- D:\Anaconda\envs\MLlabtestenv\python.exe
cachedir: .pytest_cache
rootdir: D:\ML-Labs\BSSE23105_B_ML_LAB11
plugins: anyio-4.13.0
collected 18 items

lab11_tests.py::test_load_data_returns_dataframe PASSED [ 5%]
lab11_tests.py::test_preprocess_target_not_in_X PASSED [ 50%]lab11_tests.py::test_preprocess_y_contains_correct_values PASSED [ 55%]
lab11_tests.py::test_preprocess_target_not_in_X PASSED [ 50%]lab11_tests.py::test_preprocess_y_contains_correct_values PASSED [ 55%]
lab11_tests.py::test_preprocess_target_not_in_X PASSED [ 50%]lab11_tests.py::test_preprocess_y_contains_correct_values PASSED [ 55%]
lab11_tests.py::test_preprocess_X_is_numeric PASSED [ 61%]
lab11_tests.py::test_preprocess_no_missing_values_in_X PASSED [ 66%]
lab11_tests.py::test_starter_home_status PASSED [ 72%]
lab11_tests.py::test_starter_home_message PASSED [ 77%]
lab11_tests.py::test_starter_predict_endpoint_exists PASSED [ 83%]
lab11_tests.py::test_starter_predict_accepts_json PASSED [ 88%]
lab11_tests.py::test_starter_predict_rejects_missing_body PASSED [ 94%]
lab11_tests.py::test_api_predict_missing_features_key PASSED [100%]

===== 18 passed in 10.64s =====
(MLabtestenv) PS D:\ML-Labs\BSSE23105_B_ML_LAB11> []
```